

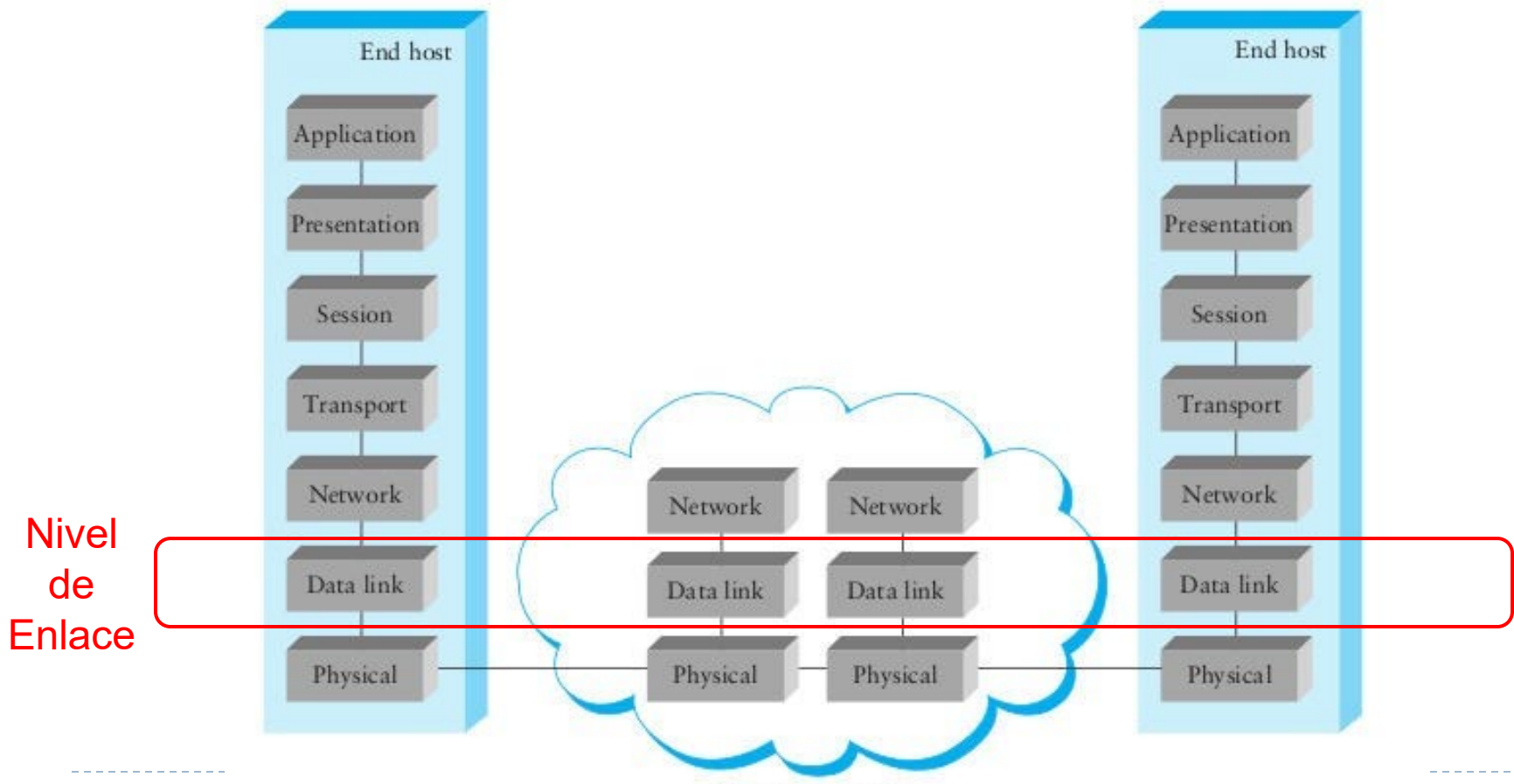


Enlaces punto a punto

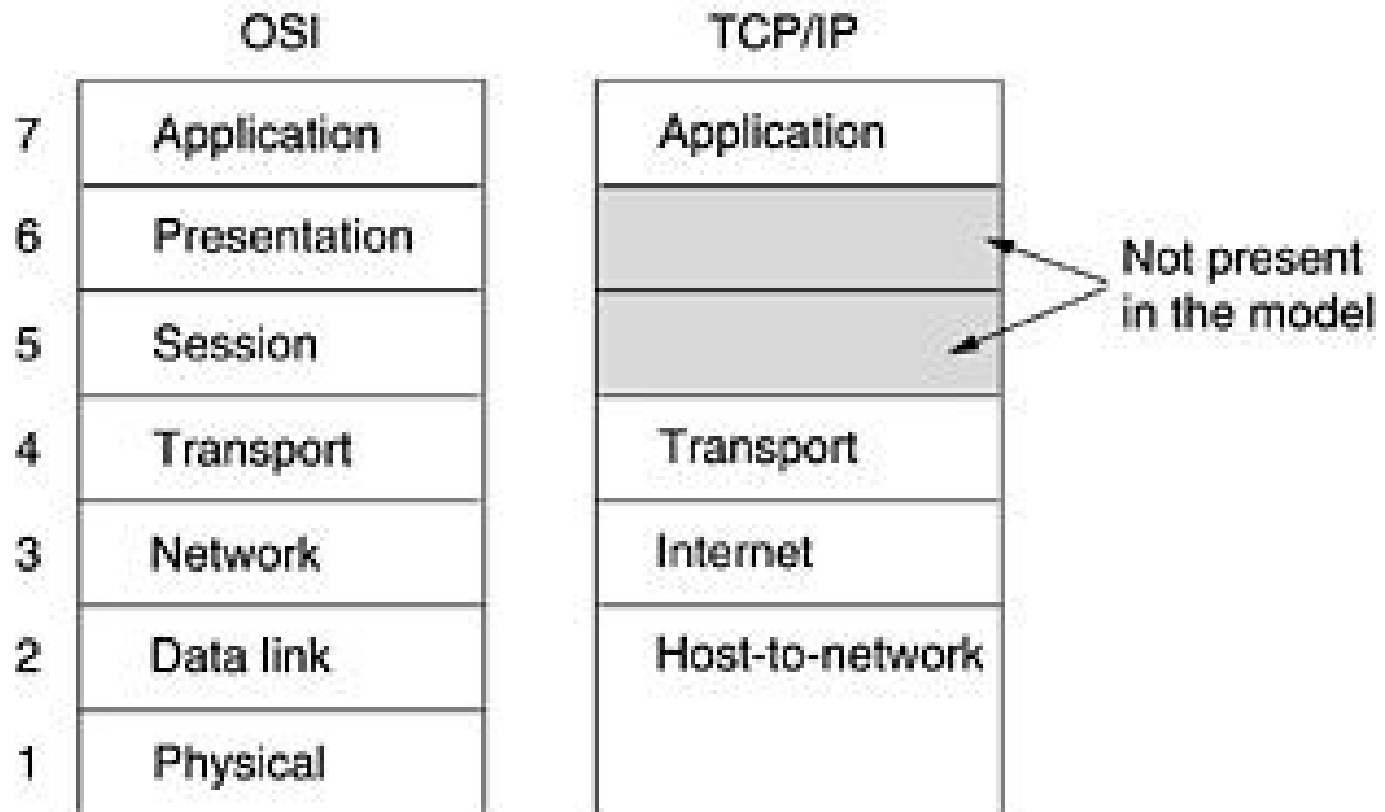
Nivel de Enlace

Paradigma de capas

- Las comunicaciones se dan en capas que se brindan servicios entre sí

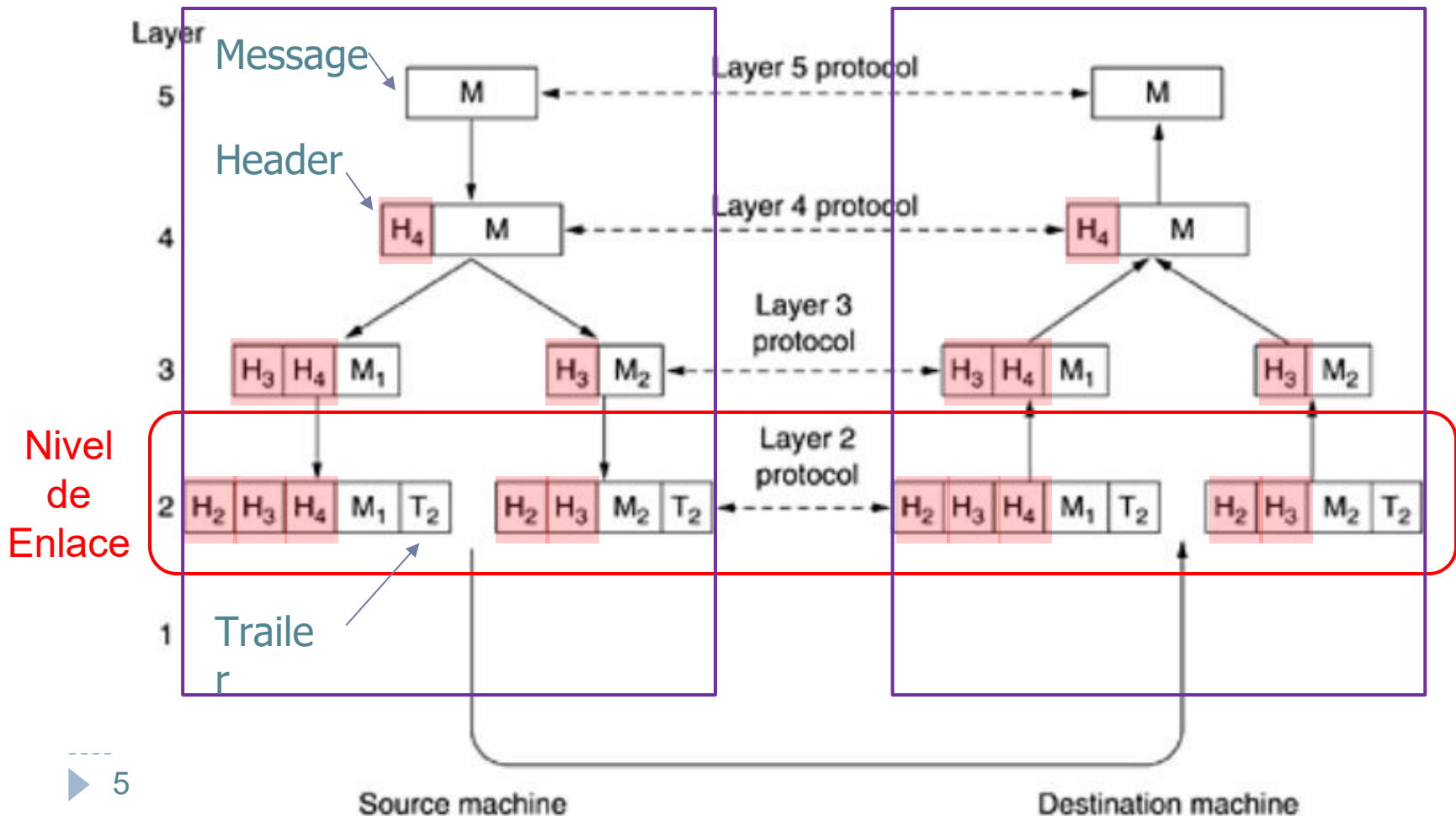


OSI-ISO vs Internet (TCP-IP)



Paradigma de capas

- ¿ Cada nueva capa implica el **agregado de información de control** en forma de **encabezados (Headers)**



Conceptos de Nivel de Enlace

- ❑ Tenemos un “caño” serial (no hay desordenamiento)
- ❑ Pero: sujeto a **ruido** y **fallas**
 - ❑ Lo que se recibe **puede no ser** lo que se envió:
“error de transmisión” (algo falló en el Nivel Físico)
- ❑ Objetivos:
 - ❑ Proveer servicio a la capa superior
 - ❑ **Confiabilidad** . ¿Confiable o no confiable?
 - ❑ **Control de Errores** . ¿Se produjo algún error?
¿Qué hacemos con los errores?
 - ❑ **Control de Flujo** . Más adelante: en Nivel de Transporte.
- ❑ Estrategia:
 - ❑ Encapsulamiento o “Framing”
 - ❑ Encapsular los bits de Mensaje en **Frames**
 - ❑ Agregar **Información de Control**

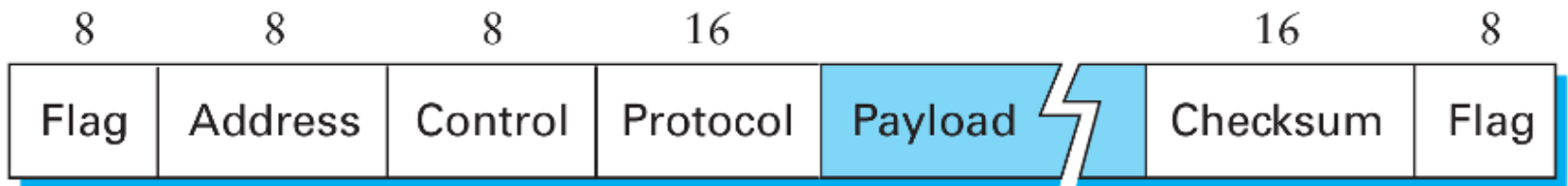
Encapsulamiento (Framing)

❓ ¿Cómo se separan los frames en un **tren de bits**?

❓ Opciones:

- ❓ Largo fijo
- ❓ Largo especificado en el encabezado
- ❓ Delimitadores de frame (con bit-stuffing)

❓ Ejemplo de protocolo PPP:



Tipo de Servicio

❑ Sin conexión y sin reconocimiento

- ❑ Los datos se envían sin necesidad de saber si llegan con errors o no.

❑ Sin conexión y con reconocimiento

- ❑ Los datos se envían y se asegura la correcta recepción sin errores mediante el **aviso explícito** (ACKs).

❑ Orientado a conexión

- ❑ Además de asegurar la ausencia de errores en la recepción de los datos, se mantiene un **estado de conexión** (una “**sesión**”).

Detección y Corrección de errores

❑ Redundancia:

❑ m bits (**datos**) + r bits (**redundancia**) = n bits (**codeword**)

❑ Definiendo:

❑ d la mínima *Distancia de Hamming* entre todas las **codewords** de un código.

❑ e la cantidad de bits erróneos en una transmisión dada

❑ Necesitamos:

❑ $e + 1 \leq d$ para poder **detectar**

❑ $2e + 1 \leq d$ para poder **corregir**

❑ Para la **confiabilidad**

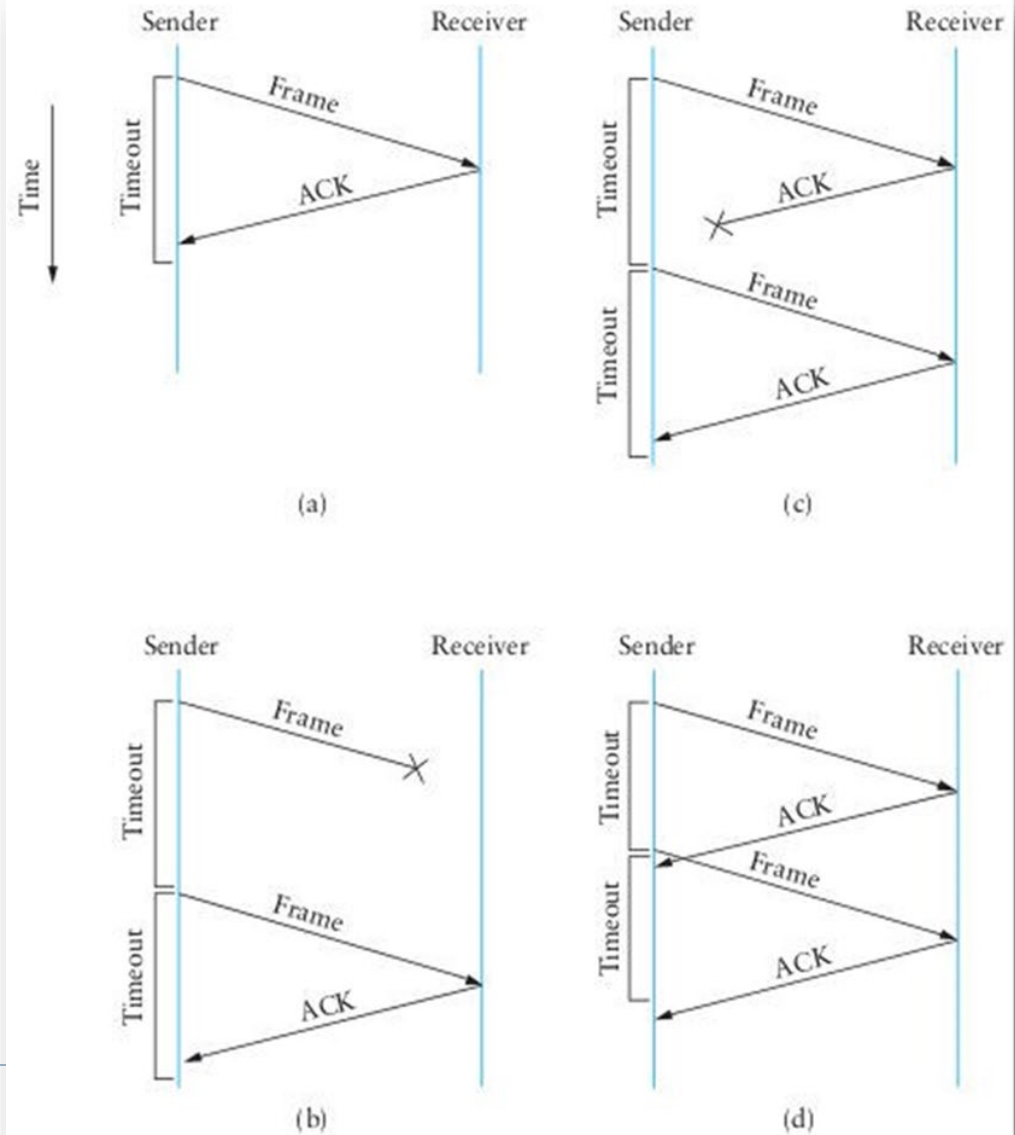
❑ Surge la necesidad de poder efectuar **retransmisiones**

❑ **Implícitas** (cuando ocurre un **time-out** se asume que el dato se perdió)

❑ **Explícitas** (**mensajes de control** específicos para pedir reenvío de datos)

Transmisión Confiable: Stop & Wait

- ❑ Cada **Frame** debe ser reconocido por el receptor.
- ❑ Problema de las **reencarnaciones**
 - ❑ Surge la necesidad de **secuenciar** (numerar unívocamente)
 - ❑ En **Stop & Wait** se necesita **secuenciar al menos 2 frames**.
- ❑ Existe un **tiempo de bloqueo** a la espera de confirmaciones.



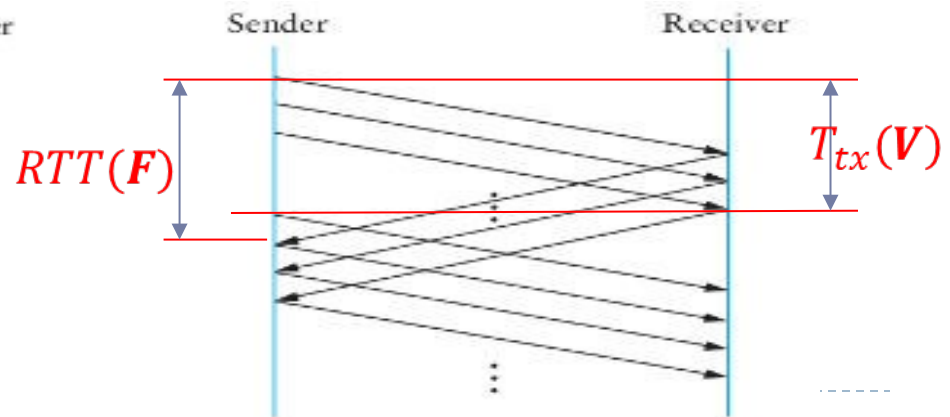
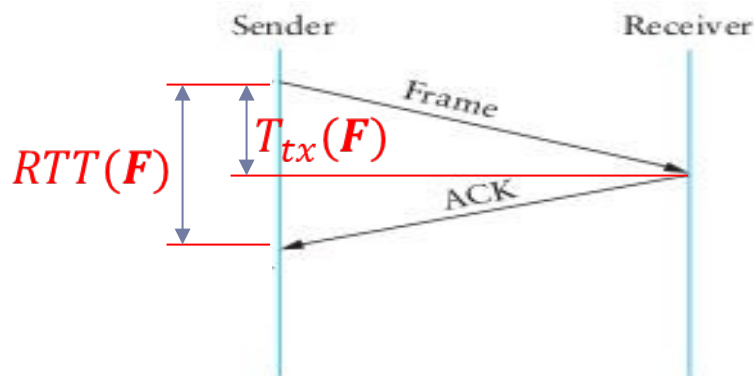
Eficiencia de protocolo

- ¿Cuánto *tiempo se está transmitiendo* con respecto al *tiempo que se está esperando* por las confirmaciones?

$$\eta_{proto} = \frac{T_{tx}}{RTT(F)}$$

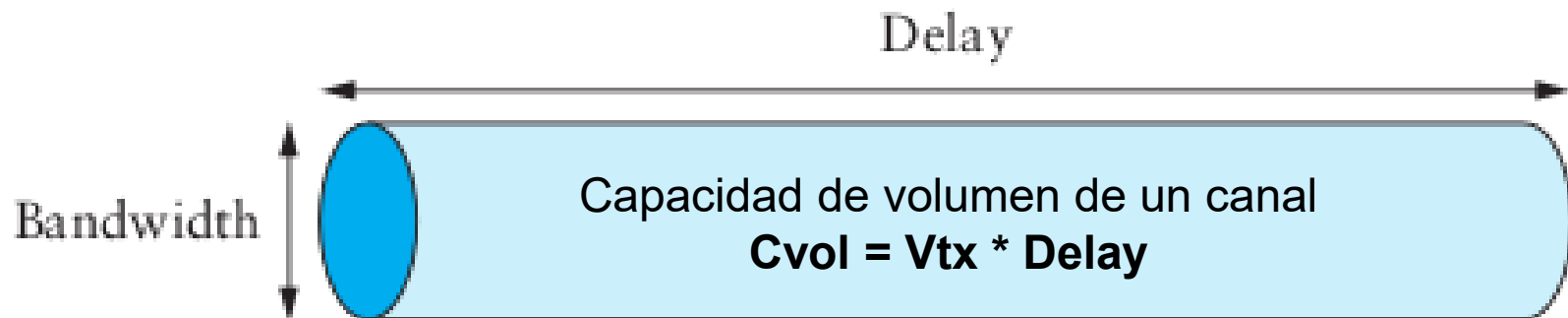
- ▶ **Aumentar la eficiencia** es estar el menor tiempo posible bloqueado esperando
- ▶ Estrategia: Enviar “**varios Frames** seguidos, sin recibir ACKs”
 - ▶ Concepto de “**Ventana** de frames” :

$$\eta_{proto} = \frac{T_{tx}(V)}{RTT(F)}$$



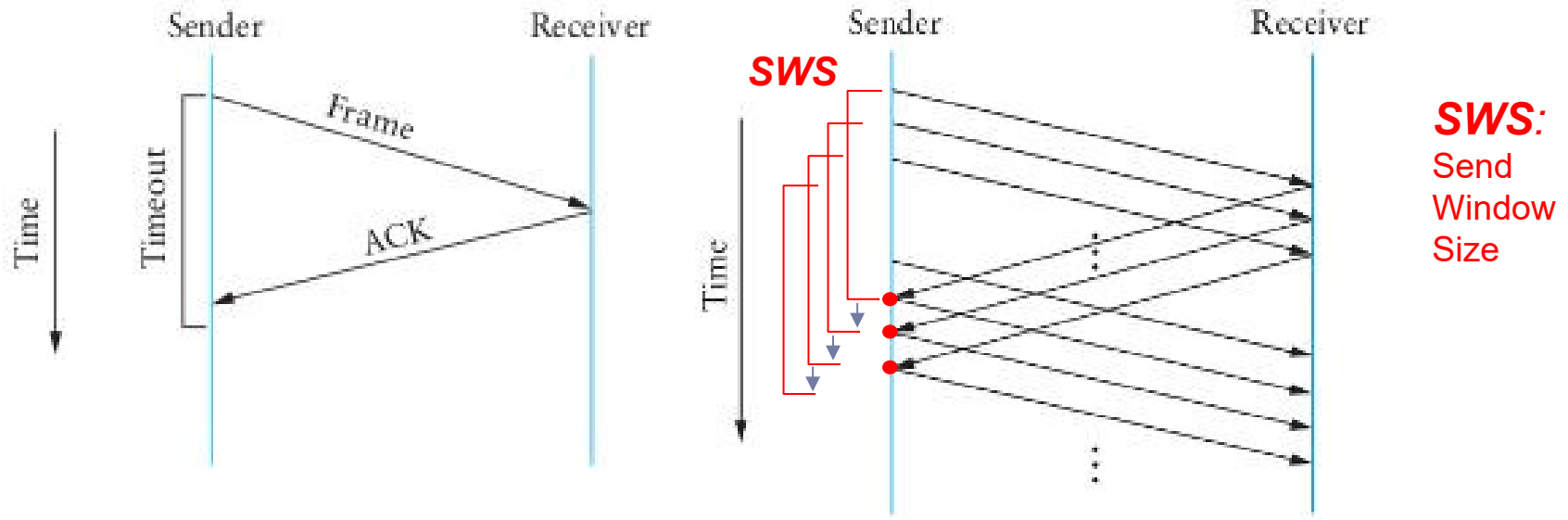
Capacidad de volumen de un canal

- ❑ Multiplicando la **Velocidad de Transmisión V_{tx}** por el **Delay** se obtiene la cantidad de bits que “entran” en un canal.
- ❑ Para aprovecharlo mejor, deberíamos calcular cuántos bits entran en el canal hasta que llega el primer ACK (es decir, usando **$2 * \text{Delay} = \text{RTT}$**)



Transmisión confiable: Sliding Window

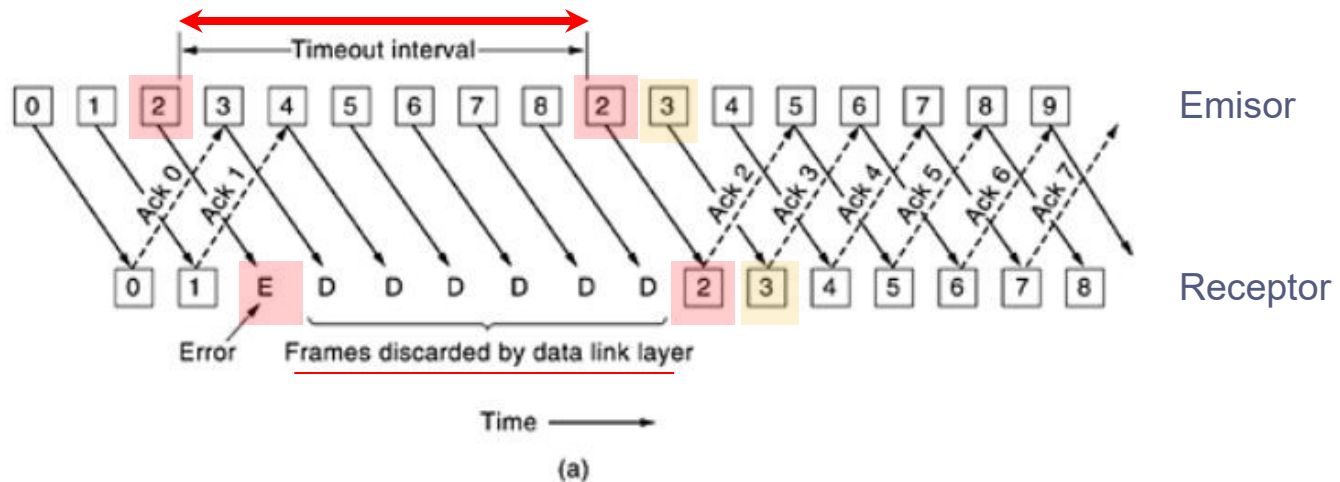
- Motivación: “mantener lleno el canal”



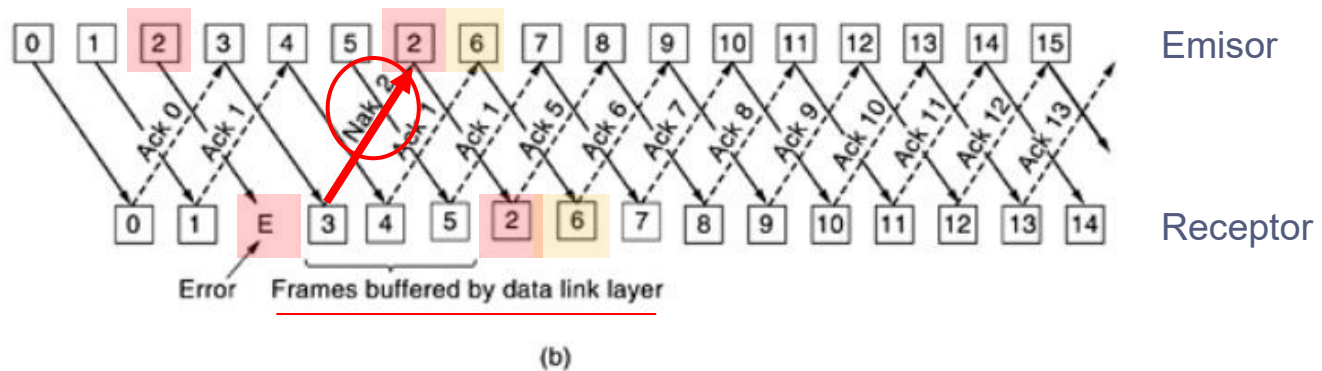
- ▶ Ventana de emisión: $SWS = \frac{V_{tx} \cdot RTT}{|Frame|}$ frames
- ▶ Enviar nuevo frame siempre que se verifique:
 $\text{ÚltimoFrameEnviado} \leq \text{ÚltimoFrameReconocido} + SWS$

Transmisión confiable: ACKs Selectivos

ACKs
acumulativos



ACKs
selectivos
(tipo: **Negative ACK**)



$$\text{Ventana de recepción: } RWS = \begin{cases} SWS & \text{si hay } \textit{Selective ACK} \\ 1 & \text{si no} \end{cases}$$

Transmisión confiable: Sliding Window

▣ Resumiendo:

- ▶ Debemos aumentar la ventana de emisión para aprovechar mejor el canal
- ▶ Ventana de **emisión**: $SWS = \frac{V_{tx} \cdot RTT}{|Frame|}$
- ▶ El receptor puede o no almacenar ("bufferear") dependiendo del esquema de ACKs
- ▶ Ventana de **recepción**: $RWS = \begin{cases} SWS & \text{si hay Selective ACK} \\ 1 & \text{si no} \end{cases}$
- ▶ Y para distinguir reencarnaciones:
 $\#frames \text{ unívocamente identificables} \geq SWS + RWS$

Delay (retardo total)

❑ T_{prop} = retardo de propagación

- ❑ Desde unos pocos microsegundos hasta a cientos de milisegundos.
Significativo para enlaces muy distantes.

❑ T_{trans} = retardo de transmisión

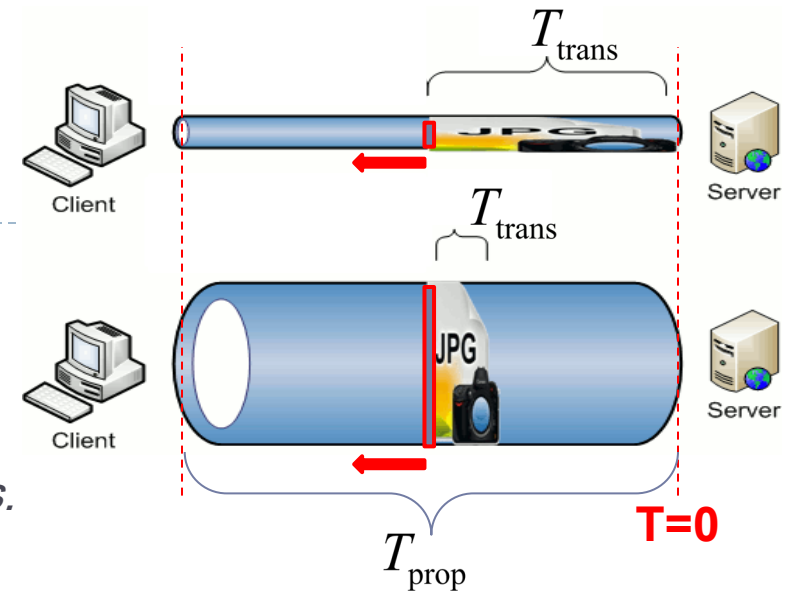
- ❑ = $\frac{\text{Tamaño Trama}}{\text{Velocidad de transmisión}}$
Significativo para enlaces de baja velocidad (o tramas muy grandes)

❑ T_{encol} = retardo de encolamiento

- ❑ Depende de la congestión. Desde nulo hasta gigante.

❑ T_{proc} = retardo de procesamiento

- ❑ Normalmente unos pocos microsegundos o menos.



$$Delay = T_{\text{total}} = T_{\text{prop}} + T_{\text{trans}} + T_{\text{encol}} + T_{\text{proc}}$$